Self-Propelled Arm System (S.P.A.S.): An Autonomous Can-Collecting Robot on a Monocular Tracked Platform

Filip Chekalkin Huaijing Hou Spaska Janeva Kamen Kanev
Giorgi Khundadze Xiaosheng Tan Stoyan Vlaev Weiye Yuan
Embedded Systems, Cyber-Physical Systems and Robotics
Technical University of Munich, Campus Heilbronn, Germany

I. Problem Statement

Indoor spaces accumulate small litter, and drink cans are among the most common items. Collecting them is repetitive, low-risk work and therefore a reasonable target for a small mobile manipulator. It is also a useful teaching problem because the robot must solve perception, navigation, decision-making, and manipulation simultaneously, and any failure in one is immediately visible.

The mission is: search for a can, drive to it while keeping it in view, stop within reach, grasp and lift it, find a bin carrying a fiducial marker, drive to the bin, release the can, and either resume searching or report completion.

What makes the problem interesting is the hardware budget. The platform carries one low-resolution RGB camera and nothing else: no stereo pair, no depth camera, no range finder, no inertial unit the software reads, and no wheel encoders. The tracked base accepts a direction and a normalised speed and returns no measurement at all. The on-board computer is a Jetson Nano with 4 GB of memory shared between processor and graphics unit.

Scope of this report. The project also built a monocular depth capability and several exploratory tools around it. None of that is on the control path of the delivered system. In the shipped configuration both approach stop criteria are appearance-based, obstacle avoidance is disabled, and grasp verification is disabled, so nothing decisive consults depth; the depth network's only remaining consumer is coarse map bookkeeping that by construction cannot trigger an action. The archived end-to-end mission analysed in Section VI in fact ran with the depth network disabled outright. This report therefore describes only the live runtime, and every claim below concerns a purely appearance-based control loop. The complete implementation is available in the project repository [1].

II. System Overview

The robot is built on a Hiwonder JetTank [8], a tracked chassis carrying a Jetson Nano, a serial-bus servo arm and a forward-facing camera. Table I lists the components, and Fig. 1 shows the platform and the two objects it has to recognise.

TABLE I
Hardware components

Component	Function
JetTank chassis	Tracked base. Accepts normalised speed commands only; returns no measurement.
Jetson Nano (4 GB)	Runs all perception, planning and control.
RGB camera	The only exteroceptive sensor, 320×240 pixels, horizontal field of view $\approx 60^\circ$.
Arm and gripper	Five serial-bus servos over a TTL chain. Position is readable; grip force is not.
AprilTag	80 mm `tag36h11`, identifier 0, mounted on the bin.

The most useful way to characterise this hardware is by what it reports back, because that asymmetry determines the software design.

The base is a pure actuator. No encoders, no current sensing, no inertial measurement. Every base parameter is an empirical constant obtained by driving the robot and recording the results, and nothing in the running system can verify those constants.

The camera is the only sensor. Every observation about the outside world passes through one device. If it is saturated, blurred or pointed the wrong way, the robot is blind, and the design must survive that rather than prevent it.

The gripper is force-blind. The servos accept an angle and a speed and can report position, which is the one place the system has genuine proprioception. They do not expose grip force or motor current, so the gripper closes to a commanded angle without knowing whether anything is inside it.

The software exploits the one asymmetry available: arm motions are verified by polling servo positions until they stop changing, whereas base motions can only be verified by looking at the camera again.

III. Perception on the Control Path

Two detectors run on the robot, and between them they supply every measurement the controller uses.

Can detection uses an SSD-MobileNet-V1 model [4], [5] at 300×300 input resolution with a single foreground class, exported to ONNX and executed through the

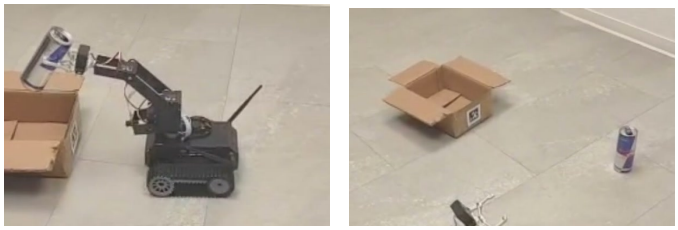

Fig. 1. The platform carrying a grasped can (left) and the two target objects in the arena (right). The can is a 250 ml slim aluminium can, 53 mm in diameter and 133 mm tall, weighing about 11 g empty. The bin is a cardboard box roughly 5 cm tall with the printed `tag36h11` marker on its side. Both photographs are frames from the recorded hardware demonstration.

`jetson-inference` `detectNet` interface [7] at a confidence threshold of 0.20 and a clustering overlap of 0.30. This architecture was chosen over the YOLO model [2], [3] used for offline dataset work because it is the path NVIDIA supports natively on the Nano; the trade is quantified in Section VI. The detector returns a bounding box, from which the controller derives two scalars: the normalised horizontal error and the normalised box height. Fig. 2 shows both the raw detector output and the same two scalars as they appear in the onboard telemetry overlay during an approach.

Bin detection locates an 80 mm `tag36h11` AprilTag [6] with identifier 0 using the `pupil_apriltags` library, with OpenCV's ArUco module as a fallback backend. Beyond the identifier, the detector computes three descriptors from the four corners, and these are what the docking controller consumes:

- the *normalised tag height*, which grows as the robot approaches and serves as the distance proxy;
- the *vertical edge error*, the relative difference between the projected lengths of the left and right edges, $e_{\mathrm{v}} = (\ell_r - \ell_l)/\frac{1}{2}(\ell_r + \ell_l)$, which is zero when the camera faces the tag squarely and changes sign with the viewing angle;
- the *maximum corner angle error*, the largest deviation of an interior corner from $90°$, which gates how much the other two should be trusted.

A metric pose is available by solving the perspective-n-point problem if a calibration file is supplied, but the delivered controller does not use it. The three descriptors above need no calibration, because the focal length cancels in the ratio, and calibration is one more artefact that can go stale. Fig. 3 shows the tag detection at close range, where the docking controller reads a tag height of 0.186 and a horizontal error of -0.055 from the same four corners.

IV. Mission Control

The mission is executed by an explicit event-driven finite state machine with fifteen states, shown in Fig. 4. Initialisation starts the camera and parks the arm. Planning selects the next target: a remembered can, a fresh search, a patrol leg, the bin, or completion. Searching, aligning, approaching and final verification are the four visual stages described in Section V. Finalising runs either the grasp sequence or the release sequence. An intermediate state absorbs timeouts, and two terminal states end the run.

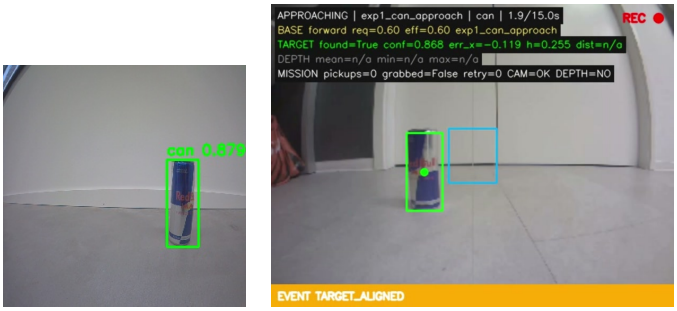

Fig. 2. Can detection. Left: the detector's own output on a 300×300 frame, with the confidence score. Right: the same detection inside the running mission, with the telemetry overlay reporting the state (`APPROACHING`), the commanded base motion, and the two scalars the controller actually uses, $e_x = -0.119$ and a normalised box height of 0.255. The entire perception stack operates at 320×240.

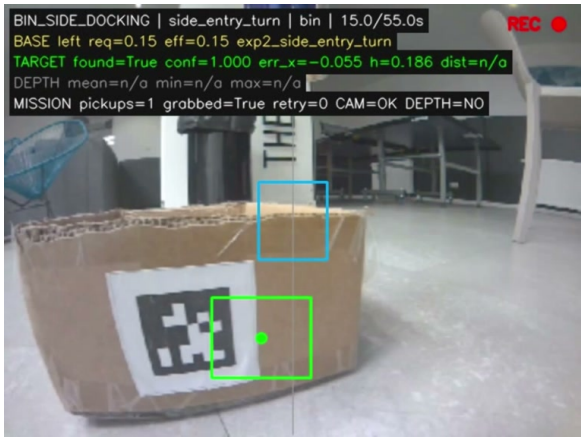

Fig. 3. AprilTag detection during bin docking. The green box marks the detected `tag36h11` marker; the overlay reports a tag height of 0.186 against the 0.15 stop threshold and a horizontal error of -0.055. Note also `grabbed=True` and `DEPTH=NO`: the run carried a can and used no depth information.

Transitions are declared as an explicit table of (state, event) pairs, and any pair not in the table raises an error rather than silently doing nothing. This is deliberate: an undeclared transition is a specification bug, and it is far easier to find when the program stops than when the robot quietly continues in an unexpected state. It also makes the transition relation a small finite object that can be reasoned about directly. Two useful properties follow from reading it. The finalising state has exactly two incoming edges, both requiring the target-stable event, so a grasp can only follow several consecutive well-aligned frames. The map-navigation state has four outgoing edges, and none of them reaches finalising, so the dead-reckoned position can never, by itself, trigger a physical action.

Failure handling is uniform. Any state that exceeds its time or step budget emits a timeout, which is routed to the intermediate state; that state increments a retry count and either replans or, once the limit is exhausted, fails. If something goes wrong unexpectedly, the system switches to a failure state. No

matter what happens, the run function always handles cleanup in a `finally` block: the base is stopped, the arm moves back to its safe position, and the camera is released. Cleanup can safely run more than once, thanks to a lock that prevents any issues if it gets called repeatedly during an interrupted mission.

All tunable values are stored in two JSON files, organised by decision type instead of by module. One file covers topics such as which paths to take, simulation flags for the hardware, and how many times the system should retry if something fails. The other file is where all the values discovered from trial and error go—speeds, timeouts, tolerances, thresholds, and the different arm positions. When it is time to use these settings, both files are combined in a set order, and any command-line changes take priority at the end. Speeds are referenced symbolically, for example, `turn.slow`, and resolved against a small profile table, so re-tuning the platform does not mean editing dozens of call sites.

Each hardware device has an independent simulation flag, and all three default to simulated, so the complete state machine can be exercised on a laptop without a robot attached. On the robot, devices are enabled one at a time, with the arm last. This is the single most effective safety measure in the project because the arm is the component most likely to damage itself.

V. Navigation and Manipulation

A. The Four Visual Stages

Can navigation and bin navigation share four stages with different parameters and detectors. For a detection with centre x_c in an image of width W, the normalised horizontal error is

$$e_x = \frac{x_c - W/2}{W} \in [-0.5,\ 0.5], \tag{1}$$

negative when the target lies left of centre. This single scalar drives every steering decision in the system.

During the **searching** phase, the system rotates in a designated direction until either the detector identifies a target or the state times out. In the **aligning** phase, the base turns toward the target until the absolute horizontal error ($|e_x|$) is within a specified tolerance. If the target is lost during this process, the base halts immediately, and the lost-frame counter is incremented. When this counter reaches a predefined threshold, the stage concludes by reporting the target as missing, preventing unnecessary rotation without a target.

Stopping immediately on loss matters, because a rotating robot that has lost its target will quickly turn past it. **Approaching** re-detects every frame, steers when $|e_x|$ exceeds a tolerance and otherwise drives forward, stopping when the target's normalised bounding-box height exceeds a threshold: 0.36 for the can and 0.15 for the bin. Apparent size is used as the distance proxy because it depends only on the detector. **Final verification** repeats alignment with a tighter tolerance and requires several consecutive frames to pass before the mission commits.

Because the base returns nothing, the system also keeps a deliberately approximate spatial memory, integrating the commands it issued to estimate its own pose and to remember roughly where cans were seen. Its purpose is attention, not localisation: a visible target always overrides it, and arriving at a remembered coordinate only triggers a visual search there. Section VI explains why nothing stronger would be defensible.

B. Grasping

The arm follows a sequence of predefined positions, in place of real-time inverse kinematics computation. Since the task parameters remain consistent—the same object, a flat surface, and a fixed approach distance—a general kinematic solution would increase complexity without offering substantial advantages for this specific scenario. The sequence is: pause for the base to settle; move to `arm_down` and wait for the servos to actually arrive; drive the base slowly forward for a fixed time, pushing the can into the open gripper; hold still; close to `grab`; lift to `carry`. Fig. 5 shows the four decisive moments of that sequence as the robot saw them.

Two steps deserve comment. The settle wait is the one place with real feedback: rather than sleeping for a fixed duration, the controller polls each servo's reported position until consecutive readings differ by less than a small threshold for several samples, with a hard timeout. If the arm has not settled, the base push is blocked outright.

If the robot moves forward before the gripper has finished lowering, it risks pushing the can away instead of picking it up. This technique, pushing the can into the gripper rather than closing the gripper around a stationary can, is somewhat unconventional, but it suits the hardware's constraints. The arm's reach is short, and the gripper's closure is somewhat imprecise, so the robot first lowers the open gripper and then moves forward to let the can slide inside. This approach avoids the need for fine manipulation and instead makes use of the base's more reliable movement.

The cost is that it works only for a light object on a low-friction floor.

VI. Results

A. Detector Accuracy

The deployed SSD-MobileNet-V1 model was evaluated against the custom model it replaced, on frozen photo and video sets captured by the robot. It reaches $F_1 = 0.839$ on photographs and 0.924 on video at its 0.20 operating threshold, against 0.899 and 0.945 for the previous model at its own 0.50 threshold. The deployed model is therefore measurably weaker, by about six points on photographs and two on video, and was still chosen because it removes a custom post-processing layer and a custom inference backend from the robot in favour of the vendor-supported path. On a fixed deadline, a slightly less accurate detector that is reliable to build and deploy is worth more than a better one that is fragile. The ONNX export was verified numerically: outputs differed from the original network by at most 4.4×10^{-7}, so the gap is a property of the architecture, not the conversion.

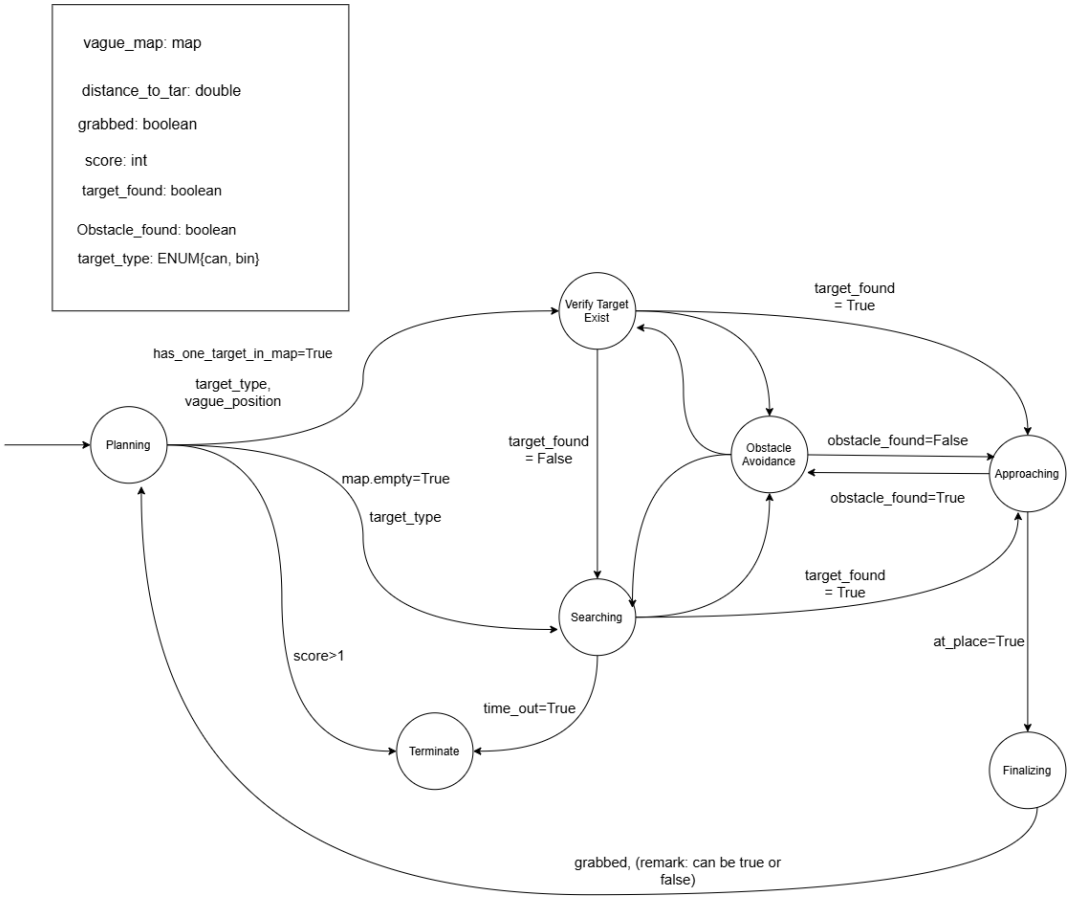

Fig. 4. The mission state machine. Guards such as `target_found` and `grabbed` are fields of a shared mission context. The implementation refines this graph, notably by splitting approach into a coarse stage and a final verification stage, but preserves its structure.

For reference, the offline YOLO11n model used for dataset iteration achieved a mean average precision of 0.995 at an intersection-over-union threshold of 0.5 on its validation split, with the training behaviour shown in Fig. 6. That figure should be read sceptically: the dataset is small, single-class and drawn from one room, so it describes how easy this detection problem is in this arena rather than how good the detector is.

B. End-to-End Mission

Table II reconstructs an archived successful mission executed on hardware, taken from its log. The run used only the camera, with the depth network disabled. Fig. 7 shows the final moment of such a run on the physical platform.

Across the run, the can detector was invoked 98 times and the tag detector 41 times, with confidence scores from 0.558 to 0.942. Three observations follow. Alignment dominates the time budget: 72 steps were needed to remove an initial error of -0.388, because each corrective pulse is deliberately small. The visual stop criterion behaved as designed, with box height growing almost monotonically and the stage halting on the first frame past threshold; the one non-monotonic step, a dip from 0.204 to 0.196, is the detector jitter such a threshold must tolerate. Roughly twelve seconds of the run are spent deliberately not moving, in the settle wait, the slow push and the post-push lock, which is the price of grasping reliably with imprecise hardware.

TABLE II
RECONSTRUCTION OF A COMPLETE SUCCESSFUL MISSION

Phase	**Observed behaviour**
Search can	Accepted on the first evaluated frame at confidence 0.586, $e_x = -0.388$.
Align can	72 corrective turn steps to bring $\lvert e_x \rvert$ inside the 0.15 tolerance.
Approach can	10 forward pulses; box height grew $0.176 \rightarrow 0.509$ against a 0.400 stop threshold.
Near align	15 further steps at the tighter tolerance.
Grasp	Arm settled in 4.94 s; 7.0 s push at speed 0.1; 5.0 s settle; grip and lift.
Bin phases	Tag acquired, 9 alignment steps, 35 forward pulses to a tag height of 0.590.
Release	Gripper opened, arm returned to its safe pose.
Total	**73.8 s, mission reported successful**

An archived failed run is equally instructive: it terminated after 19.6 s when the can was lost for more consecutive frames than the limit allows. The robot stopped cleanly rather than driving blindly, as the design intends.

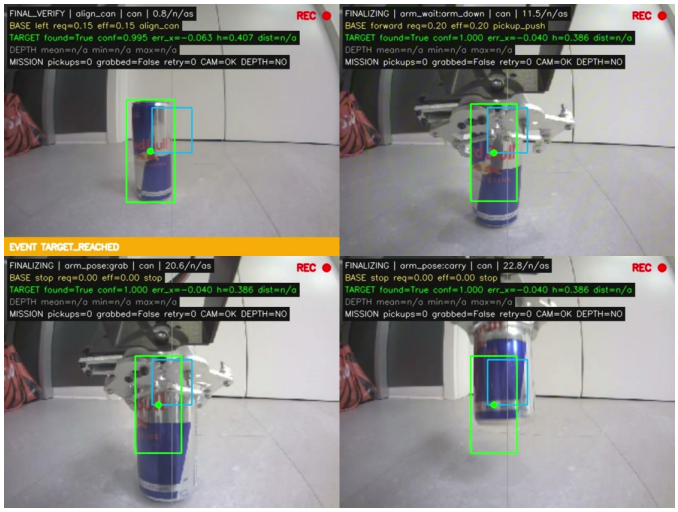

Fig. 5. The grasp sequence seen from the robot's own camera, reading left to right and top to bottom. Top left: final verification, with the can aligned to $e_x = -0.063$ and a box height of 0.407 past the 0.36 threshold. Top right: the base pushing forward at speed 0.20 with the gripper already lowered. Bottom left: the gripper closing on the can. Bottom right: the can lifted to the carry pose. The overlay's `DEPTH=NO` field confirms no depth information was used.

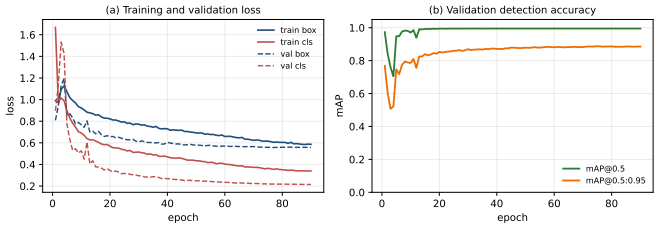

Fig. 6. Offline YOLO11n training used for dataset iteration only; this model does not run on the robot. Validation mAP@0.5 saturates near 0.995 within about twenty epochs, which reflects the narrowness of the single-room, single-class dataset more than the difficulty of the detection problem in general.

C. Measured Constraints

Table III lists the platform limits measured during development. These were the binding constraints on the design.

The turn asymmetry is the most damaging entry, and its magnitude is worth spelling out. The dead-reckoning model uses a single constant of π radians per speed-second, so the configured turn profiles predict 0.471 rad/s slow and 0.942 rad/s fast. A direction-dependent discrepancy of 1 to 3 rad/s is therefore between 1.1 and 6.4 times the modelled rate itself. This is stronger than saying the constant needs calibrating: a single scalar cannot describe a platform whose behaviour differs between two directions by more than the modelled magnitude, so the model has the wrong form. Nothing that depends on integrated heading can be trusted, which is precisely why every decisive loop in the system is closed on image appearance and why the spatial memory is never allowed to trigger an action.

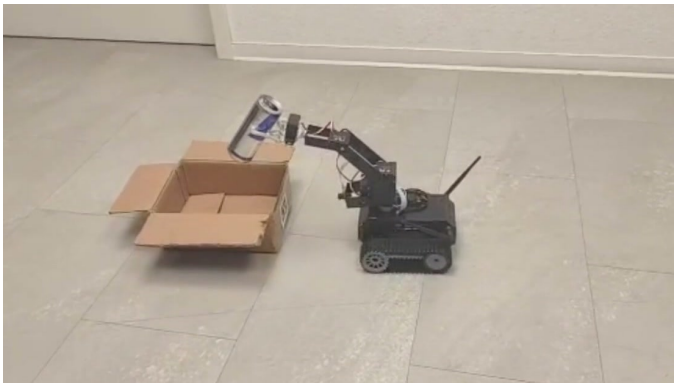

Fig. 7. The end of a complete mission on hardware: the robot has docked against the bin and holds the grasped can over it, immediately before the gripper opens and the arm returns to its safe pose.

TABLE III
MEASURED PLATFORM CONSTRAINTS

Constraint	Value
Turn rate asymmetry, left versus right, at identical commands	1–3 rad/s
Usable range of the on-board can detector	≈ 3 m
Reliable AprilTag localisation range	≈ 1 m
Sustained processing rate	10–15 frames/s
Maximum gripper reach height	≈ 20 cm

D. A Silent Failure Mode

Reading the code against the mission's intent surfaced one defect that testing had not. The grasp verification step is guarded by a configuration flag that is set to false in the shipped configuration, so the pickup is recorded unconditionally. If the push fails to capture the can, the mission context still asserts that an object is held, the robot drives to the bin, performs the release over it, and reports success with an empty gripper. Nothing downstream can detect this, because no later state re-examines the gripper.

This is the only one of the system's failure modes that is silent. Target loss, state timeouts, a stalled arm, a stale map entry and an unavailable camera frame all produce an explicit failure or a recovery. A system that fails loudly in five ways and silently in one should spend its next increment of effort on the silent one. The property is restored by enabling the flag; the deeper fix is gripper force or current sensing, which this hardware does not expose.

VII. Limitations and Conclusion

The system does what it set out to do, but the honest description is narrow. The pose estimate integrates commanded motion without correction, so its error is unbounded and its model is invalid given the measurement above. The detector was trained and validated on data from a single room with a single object type, so its metrics do not generalise. The push-based grasp depends on floor friction that was never measured, and by the finding above, the system does not notice when that assumption fails. At 10–15 frames per second, the visual

control loop is slow relative to the base, which is why motion is pulsed, and speeds are low.

Taking these limitations into account, the core design choice can be understood as a practical one. Since each decisive loop relies on image measurements rather than estimated geometry, most errors tend to degrade performance gradually rather than directly lead to failure. For example, if the map drifts, the robot needs to search a bit more; if a turn is asymmetric, it just requires some extra correction steps. With the single exception of the empty-gripper case, the system is slow and fussy but rarely confidently wrong, and on hardware with no proprioception, that is the property worth engineering for.

The contributions are the integration and the choices it forced. An explicit state machine with a declared transition table and uniform timeout handling made a fifteen-state mission debuggable and made two useful safety properties readable straight off the table. Appearance-based visual servoing allowed the system to tolerate a base with no odometry and a turn model that does not hold. The system uses an intentionally approximate spatial memory, allowing the robot to keep track of can locations without assuming it knows its precise position. With a push-based grasp, a challenging precision task is replaced with a simpler one that suits the hardware's capabilities.

The next steps for improvement are relatively inexpensive. For example, enabling grasp verification can address the silent failure mode at the cost of just one additional observation per pickup. Calibrating separate left- and right-turn constants, which the configuration already supports and currently leaves at unity, would address the largest single source of motion error. Adding one inexpensive forward-facing distance sensor would give the system its first true metric measurement, and gripper force sensing would be the proper fix for the failure mode above.

REFERENCES

[1] K. Kanev *et al.*, "Self-Propelled Arm System (S.P.A.S.)," project repository, 2026. [Online]. Available: https://github.com/kamkanev/Self-Propelled-Arm-System

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. CVPR*, 2016, pp. 779–788.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2024.

[4] W. Liu *et al.*, "SSD: Single Shot MultiBox Detector," in *Proc. ECCV*, 2016, pp. 21–37.

[5] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.

[6] E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," in *Proc. ICRA*, 2011, pp. 3400–3407.

[7] D. Franklin, "jetson-inference: Deploying Deep Learning with NVIDIA TensorRT," NVIDIA, 2024.

[8] Hiwonder, "JetTank ROS Robot Tank with Jetson Nano," product documentation, 2024.